

Modul 347

Projektwoche 3
Konfiguration & .env

Themenübersicht

- Das Problem – Passwörter im Code
- Umgebungsvariablen & .env
- .env vs .env.example vs .gitignore
- \${VAR}-Substitution in compose.yml
- os.getenv() in Python
- Docker Layer-Cache & .dockerignore
- Auftrag: Ticketboard konfigurieren

PW3 – Konfiguration & .env

■ Zielbild

- Ich kann erklären, warum Passwörter nicht direkt in `compose.yml` gehören.
- Ich kann eine `.env`-Datei erstellen und korrekt in `compose.yml` einbinden.
- Ich kann den Unterschied zwischen `.env`, `.env.example` und `.gitignore` beschreiben.
- Ich kann Variablen mit `${VAR}` in `compose.yml` substituieren und mit `docker compose config` überprüfen.
- Ich kann den Docker Layer-Cache und `.dockerignore` sinnvoll einsetzen.

Das Problem – Passwörter im Code

- Passwörter direkt im Code
 - compose.yml wird ins Git eingecheckt
 - git-Verlauf ist **permanent!**
 - einmal hochgeladen = für immer sichtbar
 - Jede Umgebung braucht eigene Werte
 - (Dev, Test, Produktion)
- Lösung: Konfiguration trennen
 - 12-Factor App – Faktor III: Config
 - Konfiguration gehört in Umgebungsvariablen und nicht in den Code.

```
# compose.yml (ACHTUNG - UNSICHER)

services:
  api:
    environment:
      DATABASE_URL: "postgresql://ticketuser:secret@db:5432/ticketdb"

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ticketdb
      POSTGRES_USER: ticketuser
      POSTGRES_PASSWORD: secret # ← Passwort im Klartext
```

.env: alle Werte an einem Ort

■ Was ist .env?

- Textdatei mit KEY=VALUE-Paaren
- Docker Compose liest sie automatisch beim Start des Projekts

■ Wichtig:

- .env NICHT ins git einchecken!
- sensible Daten bleiben lokal

```
# .env (wird NICHT committed)
```

```
POSTGRES_DB=ticketdb  
POSTGRES_USER=ticketuser  
POSTGRES_PASSWORD=geheim1234
```

```
POSTGRES_HOST=db  
POSTGRES_PORT=5432
```

```
# Zusammengesetzte URL wird in  
# compose.yml gebaut
```

.env vs. .env.example vs. .gitignore

.env

- Echte Passwörter
- **nie ins Git!**
- Nur lokal & auf Server

```
POSTGRES_PASSWORD=geheim1234
```

Ignoriert durch .gitignore

.env.example

- Nur Schlüsselnamen
- Keine echten Werte
- **Wird ins Git eingchecked**

```
POSTGRES_PASSWORD=changeme
```

Git-safe!

.gitignore

- Schützt vor versehentlichem
- Commit sensibler Dateien
- **Git-Verlauf ist permanent!**

```
.env  
__pycache__/  
*.pyc  
.venv/
```

Schutzschicht

`${VAR}` Substitution in compose.yml

- Wie funktioniert es?
 - Docker Compose liest `.env` automatisch
 - `${VARIABLE_NAME}` wird ersetzt durch den Wert aus der `.env`-Datei
- Prüfung mit:
 - `docker compose config`
 - Vgl. nächste Slide

```
# compose.yml (SICHER)

services:
  api:
    env_file:
      - .env
    environment:
      DATABASE_URL: postgresql://${POSTGRES_USER}
                    :${POSTGRES_PASSWORD}
                    @${POSTGRES_HOST}:${POSTGRES_PORT}/${POSTGRES_DB}

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

`docker compose config` – Konfiguration prüfen

- Was macht der Befehl?
 - Gibt die vollständig aufgelöste Compose-Konfiguration aus
 - Alle `${VARIABLE}` sind durch echte Werte ersetzt
 - Ideal zur Fehlersuche vor dem Start
- Wann einsetzen?
 - Nach Änderungen an `.env` oder `compose.yml`
 - Wenn Variablen möglicherweise fehlen
 - Vor dem ersten Start einer neuen Umgebung

```
$ docker compose config

services:
  api:
    environment:
      DATABASE_URL:
postgresql://ticketuser:geheim1234@db:5432/ticketdb
    ...
  db:
    environment:
      POSTGRES_DB: ticketdb
      POSTGRES_PASSWORD: geheim1234
      POSTGRES_USER: ticketuser
    image: postgres:16-alpine
    ...

# Alle ${VAR} wurden korrekt aufgelöst
```


os.getenv() in Python

- Vorher – hardcoded:
 - DATABASE_URL direkt als String im Code
- Nachher – aus Umgebung:
 - os.getenv() liest den Wert zur Laufzeit
 - Der Container erhält ihn via compose.yml
 - Kein Passwort mehr im Code

```
# VORHER ✗  
DATABASE_URL =  
"postgresql://ticketuser:secret@db:5432/ticketdb"  
  
# NACHHER ✓  
import os  
  
DATABASE_URL = os.getenv("DATABASE_URL")  
  
# Der Wert kommt zur Laufzeit aus der Umgebung.  
# compose.yml setzt ihn via:  
#   environment:  
#     DATABASE_URL: postgresql://${POSTGRES_USER}:...  
  
engine = create_engine(DATABASE_URL)
```

.dockerignore – was gehört nicht ins Image?

- Was macht .dockerignore?
 - Definiert Dateien, die beim docker build NICHT in den Build-Kontext übertragen werden
- Warum wichtig?
 - Kleineres Image (schnellerer Build)
 - Sensible Dateien landen nicht im Image!
- Merke:
 - .gitignore schützt vor Git-Upload
 - .dockerignore schützt vor Image-Upload
 - Beide Dateien sind nötig!

```
# .dockerignore

# Secrets - NIEMALS ins Image!
.env

# Git-Metadaten
.git

# Python-Cache
__pycache__/*
*.pyc
*.pyo

# Virtuelle Umgebung
.venv/

# Test-Artefakte
.pytest_cache/
tests/

# IDE
.vscode/
.idea/
```

Docker Layer-Cache – schnellere Builds

- Wie funktioniert der Cache?
 - Jeder Dockerfile-Schritt erzeugt einen Layer
 - Docker speichert Layers zwischen
 - Ändert sich nichts → Layer aus Cache
- Warum requirements.txt zuerst?
 - requirements.txt ändert sich selten
 - pip install nur bei neuen Abhängigkeiten
 - App-Code ändert sich oft → eigener Layer
 - deutlich schnellere Builds im Alltag

```
# Dockerfile - optimale Reihenfolge

FROM python:3.12-slim
WORKDIR /app

# 1) Abhängigkeiten - ändern sich selten
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 2) App-Code - ändert sich oft
COPY app /app

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

# → requirements.txt unverändert = pip install aus Cache!
# → Nur der COPY app-Layer wird neu gebaut.
```

Auftrag: Ticketboard-Projekt

■ Ziel

- Verstehen, wie xx

■ Arbeitsauftrag

- Den Arbeitsauftrag finden Sie im Repository: <https://github.com/maoedi/bbzw-m347-docker-compose-project.git> im Verzeichnis `pw03/02_project/ticketboard/README.md`

■ Sozialform

- Einzelarbeit

■ Zeitbedarf

- 45 Minuten

■ Organisation

- Erarbeiten Sie den Arbeitsauftrag in ihrem Repo!
- Bitte Repo mittels Forms mitteilen: <https://forms.gle/8TZba3rQaznnqmUp6>